SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	Shahanaj c	**Reg. No.:**	192372292
Section / Batch:	CSE AI	**Date:**	24/08/2026

Instructions to Students

• Answer the following question with proper pseudocode and logical explanation.

• Clearly specify the algorithmic approach and complexity class wherever applicable.

• Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.

• Include state-space representation, recursion steps, and pruning conditions in backtracking problems. • Neat presentation and step-by-step justification are mandatory.

Question Type	**Focus Area**	**Questions**

Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Code of Conduct

I _____Shahanaj / 192372292_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Shahanaj_____

RUBRICS FOR CALCULATION

Criteria	**Weight age**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions,	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs

		functions/modules			
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	**Wt.**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**
Problem Understanding	20%			
Algorithm Design	30%			
Use of Control Structures	20%			
Pseudocode Structure	10%			
Completeness	20%			
Total Marks (100):				

Level 1 – Beginning

Faculty In-charge	**Course Coordinator**	**Head of Department**
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CODE:

```
#include <iostream>
#include <vector>

using namespace std;

// Global variables for simplicity in this lab environment
int V; // Number of vertices
int m; // Number of colors
int graph[20][20];
int color[20];

// Function to check if it's safe to assign color 'c' to 'vertex'
bool isSafe(int vertex, int c) {
for (int i = 0; i < V; i++) {
// Check if there is an edge and if the adjacent vertex has the same color
if (graph[vertex][i] == 1 && color[i] == c) {
return false;
}
}
return true;
}

// Recursive utility function to solve the coloring problem using backtracking
```

```
bool graphColoringUtil(int vertex) {
// Base Case: If all vertices are assigned a color, return true
if (vertex == V) {
return true;
}

// Try different colors for the current vertex
for (int c = 1; c <= m; c++) {
// Check if assignment of color c to vertex is fine
if (isSafe(vertex, c)) {
color[vertex] = c; // Assign color

// Recur to assign colors to the rest of the vertices
if (graphColoringUtil(vertex + 1)) {
return true;
}

// If assigning color c doesn't lead to a solution, backtrack
color[vertex] = 0;
}
}

// If no color can be assigned to this vertex, return false
return false;
}

// Main function to solve the graph coloring problem
void solveGraphColoring() {
// Initialize all vertices as uncolored (0)
for (int i = 0; i < V; i++) {
color[i] = 0;
}

// Call utility function starting from the first vertex
if (graphColoringUtil(0)) {
cout << "Solution exists. Coloring assignment:" << endl;
for (int i = 0; i < V; i++) {
cout << "Vertex " << i << " -> Color " << color[i] << endl;
}
} else {
cout << "No solution exists with " << m << " colors" << endl;
```

```
}
}

int main() {
// Input the number of vertices
cout << "Enter number of vertices: ";
if (!(cin >> V)) return 0;

// Input the number of colors
cout << "Enter number of colors (m): ";
if (!(cin >> m)) return 0;

// Input the adjacency matrix
cout << "Enter adjacency matrix (" << V << "x" << V << "):" << endl;
for (int i = 0; i < V; i++) {
for (int j = 0; j < V; j++) {
cin >> graph[i][j];
}
}

// Execute the solver
solveGraphColoring();

return 0;
}
```